# Two Concurrency Bugs on nixnan's `print_histo_instrn` Branch: Diagnosis, Fixes, and Verification

ganesh@cs.utah.edu
(written with Claude Code)

September 24, 2026

**Abstract**

While porting `BIN_SPEC_FILE`'s adaptive threshold-doubling feature (`doublings`) from nixnan's `fp-reset` branch onto `print_histo_instrn`, two independent concurrency bugs were found and fixed in nixnan's own source, not in any user program. The first is a genuine correctness bug in the doubling mechanism itself, present in `fp-reset`'s original code: concurrent GPU threads can race on non-atomic updates to shared per-bin state, corrupting the reported threshold sequence. The second is a pre-existing, format-independent bug in how nixnan prints *any* report at all: two independent background threads (exception reports and histogram threshold reports) write to the same output stream with no synchronization, producing torn, interleaved lines whenever both are active at once. Both are fixed here, with before/after evidence from real runs against `rd_nixnan.cu`.

## Contents

**1 Context** **2**

**2 Bug 1: The Adaptive-Doubling Race** **2**
2.1 The mechanism . . . 2
2.2 The original code, and why it races . . . 2
2.3 Evidence: a deliberately worst-case stress test . . . 3
2.4 The fix: atomicCAS-gated transitions . . . 3
2.5 Verification . . . 4

**3 Bug 2: Torn Output Between the Exception and Histogram Report Threads** **4**
3.1 The mechanism . . . 4
3.2 Evidence: a real captured example . . . 5
3.3 The fix: a single, mutex-protected write per message . . . 5
3.4 Verification . . . 5

**4 Why Neither Bug Existed Before** **6**

**5 Summary** **6**

# 1 Context

`print_histo_instrn` (off `auto-nixnan`) added `record_inst`: histogram threshold reports that also name the instruction responsible, not just the format/kernel/range. A follow-up request asked for the same exponentially decaying report rate nixnan's own documentation had long described — `BIN_SPEC_FILE`'s `doublings` key — to be used directly, rather than orchestrated by hand the way `SAMPLING` has to be (no native schedule for that env var; see the main `report.tex`, Section 4). Checking the actual source on `print_histo_instrn` (and, for that matter, `main`) found that `doublings` had never been implemented there at all — only documented. The real implementation lived on a separate branch, `fp-reset`. Porting it in, and combining it with `record_inst` (which `fp-reset` never had), surfaced both bugs documented here.

# 2 Bug 1: The Adaptive-Doubling Race

## 2.1 The mechanism

A `BinCounter` watching one exponent range accumulates a running occurrence count (`count`, via `atomicAdd`) and compares it against a `threshold`. With `doublings` enabled, that comparison is against a value that itself changes over the life of the process:

- Start at `threshold = count` (the spec's starting value).
- Each time an occurrence crosses a multiple of `threshold`, report, then double `threshold` and increment `times_doubled`.
- Once `times_doubled` reaches `doubling_limit`, the *next* crossing resets every bin in that kernel's `BinArray` back to the starting `threshold` and `times_doubled=0`.

With `count:1, doublings:8`, this produces the sequence $1, 2, 4, 8, 16, 32, 64, 128, 256$, then reset — eight doubling steps from 1 to 256.

## 2.2 The original code, and why it races

`fp-reset`'s implementation of the crossing check:

```
unsigned long long int prev = bin.increment();
if (prev % threshold == 0 && prev != 0) {
    // ... send report ...
    if (bin.doubling_limit != 0) {
        if (bin.times_doubled == bin.doubling_limit) {
            reset_bin_thresholds(bins);
        } else {
            bin.threshold *= 2;
            bin.times_doubled++;
        }
    }
}
```

`bin.count` (the running occurrence counter) is shared across *every GPU thread* that ever hits this bin — it is a per-range, whole-kernel counter, not a per-thread one. So is `bin.threshold` and `bin.times_doubled`. The crossing check above is a classic check-then-act race: many threads

can read the same `threshold` concurrently, all satisfy `prev % threshold == 0` for their own distinct `prev` values, and then all attempt the state transition at once. `bin.threshold *= 2` and `bin.times_doubled++` are both plain, non-atomic read-modify-write operations — under concurrent execution, increments can be silently lost, and the reset condition can be evaluated against a stale or inconsistent `times_doubled`.

### 2.3 Evidence: a deliberately worst-case stress test

To make contention as severe as possible, FP16's *entire* valid exponent range was watched (`"f16": [[0,15]]`, rather than a narrow couple of exponents), against `rd_nixnan`'s default kernel, before any fix:

```
1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 1 1 ...
```

The threshold value visibly falls *back* from 2 to 1 without ever reaching 4 — impossible under the intended semantics (threshold only ever doubles or resets all the way back to the start; it cannot revisit 2 after having been at 2 unless a reset already happened, which would also have reset the *other* format/range bins, easily checked and not the case here). This is exactly the lost-update/stale-read signature: multiple threads independently satisfying the crossing condition at the same nominal `threshold`, racing on the non-atomic update.

### 2.4 The fix: atomicCAS-gated transitions

Both state transitions are now gated with `atomicCAS`, so that of however many threads concurrently observe the same stale `threshold`, exactly one of them performs the actual transition:

```
unsigned long long int threshold = bin.threshold; // re-read fresh per occurrence
if (prev % threshold == 0 && prev != 0) {
    // ... send report (threshold captured above) ...
    if (bin.doubling_limit != 0) {
        if (bin.times_doubled < bin.doubling_limit) {
            if (atomicCAS(&bin.threshold, threshold, threshold * 2) == threshold) {
                atomicAdd(&bin.times_doubled, 1u);
            }
        } else {
            unsigned int limit = bin.doubling_limit;
            if (atomicCAS(&bin.times_doubled, limit, limit + 1) == limit) {
                reset_bin_thresholds(bins);
            }
        }
    }
}
```

The doubling transition is gated on `threshold` itself: a CAS from the exact value a thread just observed to double that value succeeds for at most one thread per actual transition; every other thread's CAS fails harmlessly (the comparand no longer matches) and that thread does nothing further — it already reported above, correctly, for its own genuine occurrence. The reset transition is gated the same way on `times_doubled`, using `doubling_limit` itself as the expected value and `doubling_limit+1` as a one-shot "already reset" marker, so `reset_bin_thresholds` — which touches every bin, every format, in that kernel — runs exactly once per cycle rather than once per thread that happens to observe the terminal threshold.

Multiple reports *at the same threshold value* are still expected and correct under this fix (each represents a genuinely distinct occurrence crossing that same multiple, e.g. several grid points reaching the ceiling within the same or adjacent kernel launches); what the fix eliminates is corruption of the *transition* itself.

### 2.5 Verification

**Same worst-case range, after the fix.** Re-running the identical `"f16": [[0,15]]` stress spec no longer produces backward jumps; every observed value belongs to the set $\{1, 2, 4, 8, 16, 32, 64, 128, 256\}$ and cycles reset cleanly, though with heavier repetition at low levels than a narrow range shows (more threads genuinely contend for the wide range).

**A realistic (narrow) range.** FP16's smallest-two/largest-two normal exponents, `count:1, doublings:8`:

```
1 1 1 1 1 1 1 2 2 4 8 8 8 8 8 16 16 16 32 32 64 64 128 256
```

Cleanly monotonic within the cycle — repeats at one level (e.g. five reports at threshold 1, five at threshold 8) are real, correct reports of distinct grid points crossing the same threshold in close succession, not corruption.

**A second, independent run: FP32 overflowing, FP64 not.** With `count:1, doublings:10` watching only FP32's highest two exponents (`[126,127]`) at $M$=2000 (where FP32 fails and FP64 correctly stays finite — see the main report, Section 9):

| 1 | 2 | 4 | 8 | 16 | 32 | 64 | 128 | 256 | 512 | 1024 |
|---|---|---|---|---|---|---|---|---|---|---|
| 349 | 76 | 29 | 24 | 18 | 16 | 11 | 12 | 11 | 11 | 10 |

All eleven levels present, occurrence counts falling off with level as expected, cycling repeatedly for the rest of the run — see `Logs/Exp-Output-Rate-FP32-Top2Exp-M2000-start-1-double-10-times.log` for the full trace.

## 3 Bug 2: Torn Output Between the Exception and Histogram Report Threads

### 3.1 The mechanism

nixnan has two independent background threads that each print reports as they arrive over a device-to-host channel:

- `nixnan.cu`'s exception receiver (`recv_thread_fun`), printing `"error [...] detected ..."` lines.
- `fp-histogram.cu`'s threshold-report receiver, its own separate `std::thread` spawned in `tool_init()`, printing `"... bin has reached threshold: ..."` lines.

Both call `nnout()`, which returns a reference to the same process-global `std::ostream` (`g_nnout_stream`, default `std::cerr`; `nntools.cc`). Each report is assembled as a *chain* of several `operator<<` calls:

```
nnout() << "error [" << errors << "] detected in operand " << ei->operand()
       << " of instruction " << instr << " in function " << func
       << source_location << " of type " << type << std::endl;
```

Nothing serializes this chain against another thread doing the same thing at the same time. The OS and C++ runtime are free to interleave individual `<<` calls from two different threads' messages arbitrarily.

### 3.2 Evidence: a real captured example

From an actual run combining both mechanisms at once (the script `Stop_First_Error_with_History_FP16_SAMPLING8_MAXERRORS4.sh`, chosen specifically because it drives both report paths hard simultaneously):

```
#nixnan: error [infinity#nixnan: f16 bin has reached threshold: function=rd_step_fp16(__half c\
onst*, __half*, __half, __half) range=[14,15] count=64
] detected in operand 0 of instruction HFMA2 R0, R9.H0_H0, c[0x0] [0x170].H1_H1, R0.H0_H0 ; in\
 function rd_step_fp16 of type f16
```

Reading this as two separate, individually-correct messages makes it clear what happened: the exception thread had written `"#nixnan: error [" + "infinity"` and was about to write `"] detected in operand "` when the histogram thread's entire (shorter) message interleaved in the middle, completing before the exception thread resumed and finished its own line.

### 3.3 The fix: a single, mutex-protected write per message

A new helper, `nnout_line()` (`nntools.hh`/`nntools.cc`), takes a fully-assembled message string and writes it as one locked unit:

```
static std::mutex g_nnout_mutex;

void nnout_line(const std::string& msg) {
    std::lock_guard<std::mutex> lock(g_nnout_mutex);
    *g_nnout_stream << "#nixnan: " << msg;
    if (msg.empty() || msg.back() != '\n') {
        *g_nnout_stream << "\n";
    }
    g_nnout_stream->flush();
}
```

Both report sites, and `nnterminate()` (used by `MAX_ERRORS` and `max_reports` to end the program early), now build their message into a local `std::ostringstream` first, then call `nnout_line(oss.str())` once — there is no longer a multi-part `<<` chain against the shared stream from either thread.

### 3.4 Verification

The same high-contention script was rerun three times after the fix. Each full log (1,300+ lines) was scanned for the torn-line signature — any single line containing both `"error ["` and `"bin has reached threshold"`:

```
grep -c "error␣\[.*bin␣has␣reached\|bin␣has␣reached.*error␣\[" "$LOG"
```

Zero occurrences, all three runs, all four exception messages fully intact each time. The other three scripts in this experiment (`Stop_First_Error_with_History_FP16.sh`, its `_Sampled` variant, and `Exp-Output-Rate-start-1-double-8-times.sh`) were also rerun and produce identical output to before the fix — the change is purely to *how* messages are written, not what they say.

## 4 Why Neither Bug Existed Before

Both bugs are specific to mechanisms this work introduced or extended; nixnan's pre-existing code was never exposed to either failure mode, and not because anyone engineered around it — the existing designs simply never created the opportunity.

**The doubling race (§2).** Every other counting path in nixnan checks a running `atomicAdd` counter against a value that never changes for the life of the kernel: a fixed `count` threshold (plain histogram counting), or a single first-occurrence latch (exception deduplication, gated on `num_exceptions == 0` from one `atomicAdd`). Neither pattern has any shared, mutable *control* state for concurrent threads to race on — the only shared state is the monotonic counter itself, and a single `atomicAdd` already makes that safe. `doublings` is the first mechanism whose per-occurrence decision depends on state the same mechanism also writes, in a read-decide-write cycle spanning two separate fields (`threshold` and `times_doubled`) that must change together, consistently, exactly once per logical event. That is a structurally harder concurrency problem, and confirmed (Section 2.4) to need `atomicCAS`, not a plain `atomicAdd`, to close correctly.

**The output tearing (§3).** This one predates `doublings` and `record_inst` entirely — it has existed as long as `HISTOGRAM=1` and exception reporting could both be active in the same process (the default). It was not fixed here because it was newly introduced; it was fixed because this work was the first to drive both report threads hard enough, simultaneously, to make the interleaving reliably visible rather than a rare, easily dismissed anomaly.

## 5 Summary

| | Bug 1: Doubling race | Bug 2: Output tearing |
|---|---|---|
| Where | `fp-fun.cu`, `record()` | `nixnan.cu`, `fp-histogram.cu` |
| Introduced by | Original `fp-reset` port | Pre-existing (any concurrent HISTOGRAM + exceptions) |
| Root cause | Non-atomic read-modify-write on shared `threshold`/`times_doubled` | Unsynchronized multi-part `<<` chains to one shared stream |
| Fix | `atomicCAS`-gated transitions | Mutex-protected single-write `nnout_line()` |
| Verified by | Narrow- and wide-range doubling traces; FP32/FP64 run | 3× rerun of the highest-contention script, zero torn lines |

Both fixes are narrowly scoped to the specific code paths that needed them and do not change the

behavior or output format of any existing, previously-correct nixnan functionality — confirmed by rerunning every prior script in this experiment and observing identical output to before either fix.